Annullare Modifiche in Git

Una guida pratica per recuperare il controllo delle tue modifiche



© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Programma della Lezione

01

Annullare Modifiche Locali

Impareremo come ripristinare file modificati utilizzando diversi comandi Git

02

Lavorare con i Branch

Scopriremo come creare, navigare e gestire branch nel tuo repository

03

Merge dei Branch

Uniremo le modifiche da branch diversi in modo sicuro ed efficace

04

Esercitazioni Pratiche

Metteremo in pratica tutto ciò che abbiamo imparato con esercizi guidati passo-passo

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it



Annullare Modifiche Locali

Capita a tutti di fare modifiche che poi vogliamo annullare. Git offre diversi strumenti potenti per ripristinare i file allo stato precedente. Capire quando e come usare questi comandi è fondamentale per lavorare con sicurezza e fiducia.

Esploriamo i metodi principali per annullare le modifiche locali prima che vengano committate nel repository.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Metodi per Annullare Modifiche

git checkout -- <file>

Comando classico (pre Git 2.23)

Ripristina un file specifico all'ultimo commit, scartando tutte le modifiche non salvate.

```
git checkout -- nome-file.txt
```

Attenzione: Questa operazione è irreversibile. Le modifiche non salvate verranno perse definitivamente.

git restore <file>

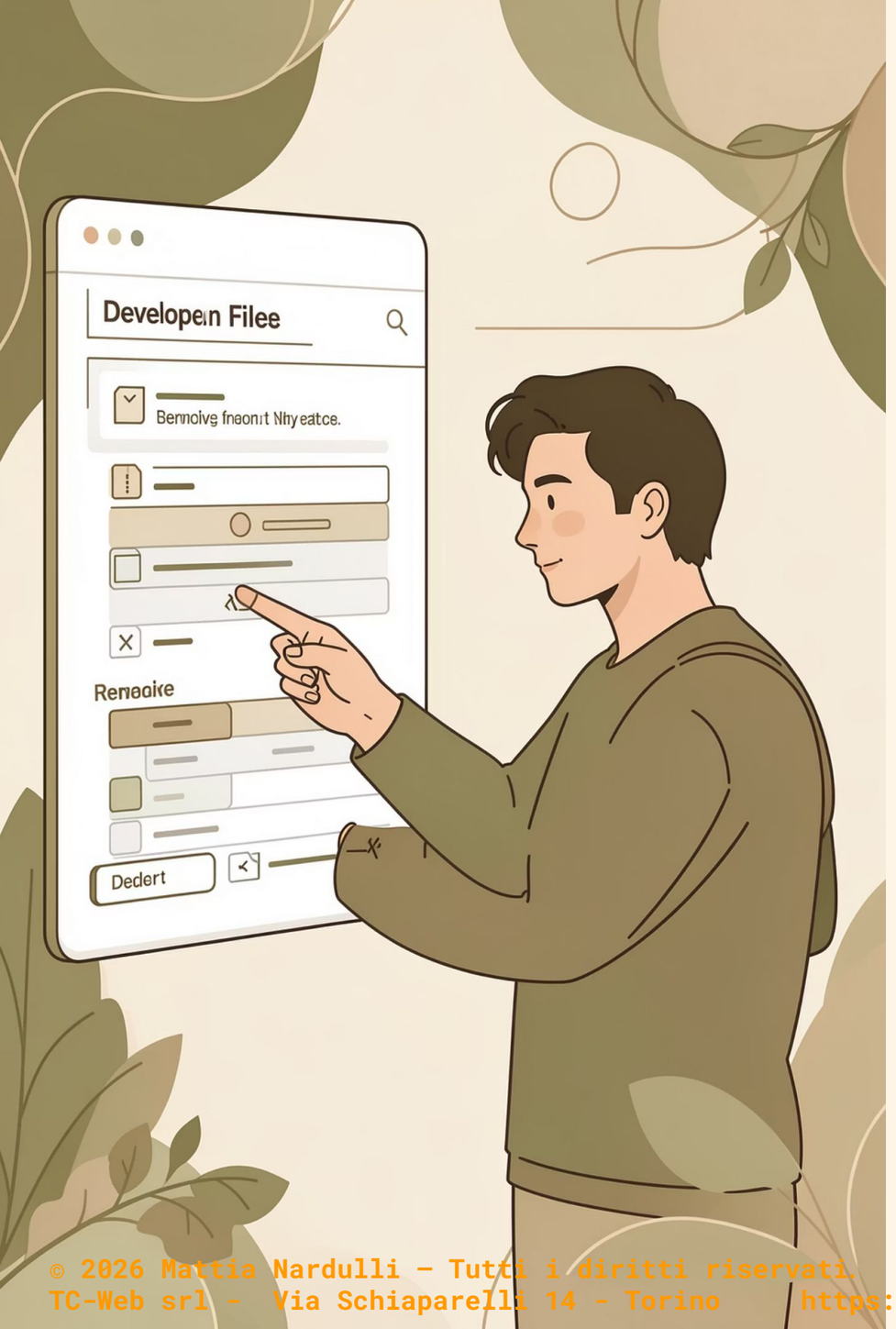
Comando moderno (Git 2.23+)

Più intuitivo e chiaro nel suo scopo. Funziona esattamente come checkout ma con sintassi più comprensibile.

```
git restore nome-file.txt
```

Best Practice: Usa git restore per progetti nuovi. È il metodo raccomandato dalla comunità Git.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it



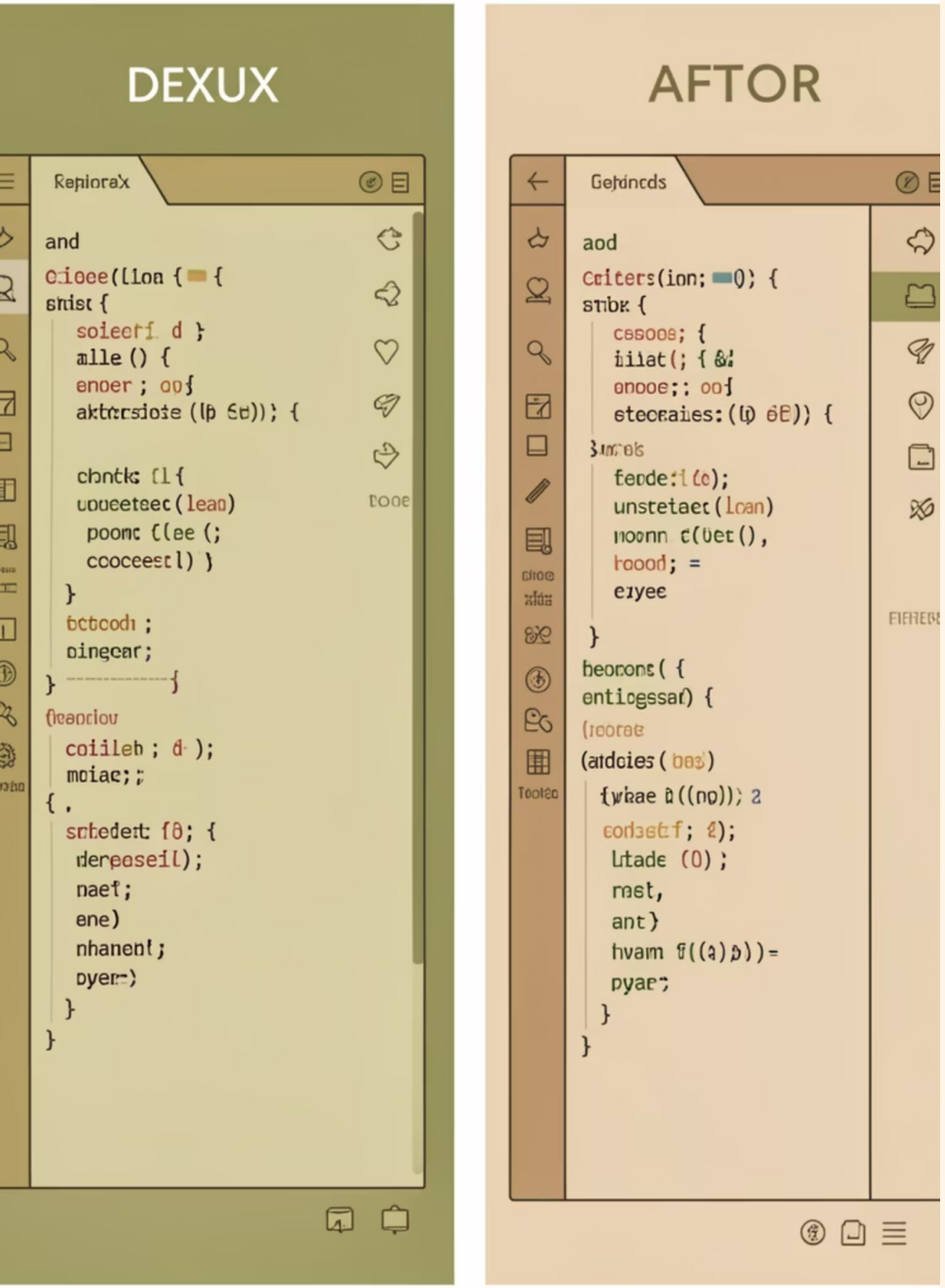
Git Restore --staged: Tornare Indietro dallo Staging

Quando hai usato `git add` per preparare i file ma ti rendi conto di aver commesso un errore, `git restore --staged` è la soluzione. Questo comando rimuove i file dallo staging area riportandoli nel working directory, mantenendo le modifiche ma senza prepararle per il commit.

1. **File modificato**
   Il tuo file ha modifiche nel working directory
2. **Git add**
   Il file entra nello staging area
3. **Git restore --staged**
   Ritorna al working directory con modifiche intatte

Attenzione: Questo comando non cancella le modifiche, le riporta semplicemente nel working directory dove puoi modificarle o ripristinarle.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it



Esempi Pratici di Ripristino

Situazione Iniziale

Hai modificato `nome-file.txt` ma le modifiche hanno causato problemi

```
git status
modified: nome-file.txt
```

Ripristino

Annulli le modifiche con restore

```
git restore nome-file.txt
```

Risultato

Il file torna allo stato dell'ultimo commit

```
git status
nothing to commit
```

Questo flusso mostra come ripristinare rapidamente un file problematico. Puoi applicare lo stesso processo a più file contemporaneamente usando `git restore .` per ripristinare tutti i file modificati nella directory corrente.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Git Revert: Annullare un Commit Pubblicato

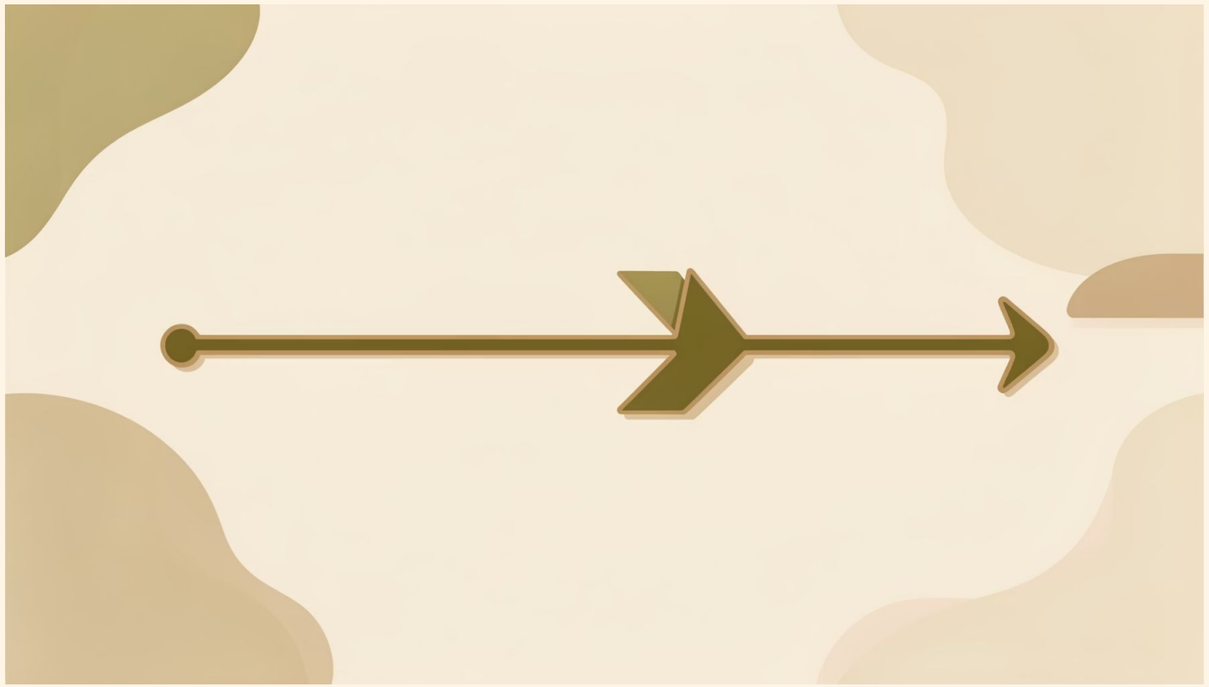
Quando usarlo

`git revert` crea un nuovo commit che annulla le modifiche di un commit precedente. È la scelta migliore quando hai già condiviso il commit con altri collaboratori.

A differenza di altri comandi, `git revert` non cancella la cronologia, ma la arricchisce con un nuovo commit che contiene l'inverso delle modifiche.

Sintassi

```
git revert HEAD
git revert abc1234
```



1 Commit A
Feature implementata

2 Commit B
Modifiche problematiche

3 Git revert
Nuovo commit che annulla B

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Git Reset: Soft vs Mixed

`git reset` è un comando potente ma pericoloso. Esistono diverse modalità, ognuna con effetti diversi sulla cronologia del progetto.

Git reset --soft

Rimuove il commit dalla cronologia ma mantiene le modifiche nello staging area. Perfetto quando vuoi modificare il messaggio del commit o aggiungere ulteriori file.

```
git reset --soft HEAD~1
```

Git reset --mixed (default)

Rimuove il commit e sposta le modifiche dallo staging al working directory. Le modifiche sono ancora presenti ma non pronte per il commit.

```
git reset --mixed HEAD~1
# Oppure semplicemente:
git reset HEAD~1
```

⚠️ Importante: Non usare `git reset` su commit condivisi pubblicamente, può causare problemi ai tuoi collaboratori.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

TC-Web Technology Consulting

Confronto Comandi di Undo

Questa tabella riassume quando usare ogni comando in base alla situazione in cui ti trovi.

Comando	Quando usarlo	Effetto sulla cronologia
`git restore`	Modifiche non aggiunte allo staging	Nessun effetto - ripristina file
`git restore --staged`	File nello staging da rimuovere	Nessun effetto - sposta indietro
`git revert`	Commit già condivisi con altri	Crea nuovo commit di annullamento
`git reset --soft`	Commit locale da modificare	Rimuove commit, mantiene staging
`git reset --mixed`	Commit locale da rifare	Rimuove commit, mantiene modifiche

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Errori Comuni da Evitare

1

Usare reset su commit condivisi

Non modificare mai la cronologia di commit già pushati su un repository condiviso. Usa `git revert` invece di `git reset`.

2

Dimenticare di specificare i file

Sii sempre specifico su quali file vuoi ripristinare. Usare `.` o `*` senza attenzione può cancellare modifiche importanti.

3

Confondere restore e reset

`restore` è per modifiche locali, `reset` è per la cronologia. Non usare `reset` come sostituto di `restore`.

4

Non verificare lo stato prima

Sempre eseguire `git status` e `git log` prima di annullare modifiche per capire esattamente dove sei.



© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Esercizio Guidato Pratico

Seguiamo insieme un esercizio per applicare quanto abbiamo imparato. Apri il terminale e crea una cartella di prova.

1. Preparazione

```
mkdir git-practice
cd git-practice
git init
echo "Ciao mondo" > file.txt
git add file.txt
git commit -m "Primo commit"
```

2. Modifiche al file

```
echo "Nuova riga" >> file.txt
git status
```

3. Esercizio A: Ripristinare

```
git restore file.txt
git status
```

Il file è tornato allo stato del commit!

4. Esercizio B: Staging e reset

```
echo "Testo modificato" > file.txt
git add file.txt
git reset HEAD~1
```

💡 Prova tu: Dopo ogni comando, usa `git status` per vedere come cambia lo stato dei file. Questo ti aiuterà a capire meglio il flusso di lavoro.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Git Branch: L'Arte di Creare Percorsi Separati

Impara a gestire il tuo codice con la potenza dei branch in Git



© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Cos'è un Branch? Una Metafora Visiva



Immagina il tuo progetto come un **albero**. Il ramo principale, chiamato **main**, è il tronco centrale. I branch sono i rami che si diramano da questo tronco, permettendoti di crescere nuove funzionalità senza danneggiare la pianta principale.
Ogni branch è un **percorso separato** della storia del tuo progetto. Puoi lavorare su nuove idee, correggere bug o sperimentare senza preoccuparti di rovinare il lavoro stabile nel ramo principale.
È come avere più copie del tuo progetto dove testare liberamente, ma con l'intelligenza di Git per unire tutto insieme quando sei pronto!

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

I Comandi Fondamentali per i Branch

Git fornisce alcuni comandi essenziali per gestire i tuoi branch. Impararli è facile!

git branch

Mostra tutti i branch del tuo progetto. Il branch attivo è segnato con un asterisco (*).

git switch

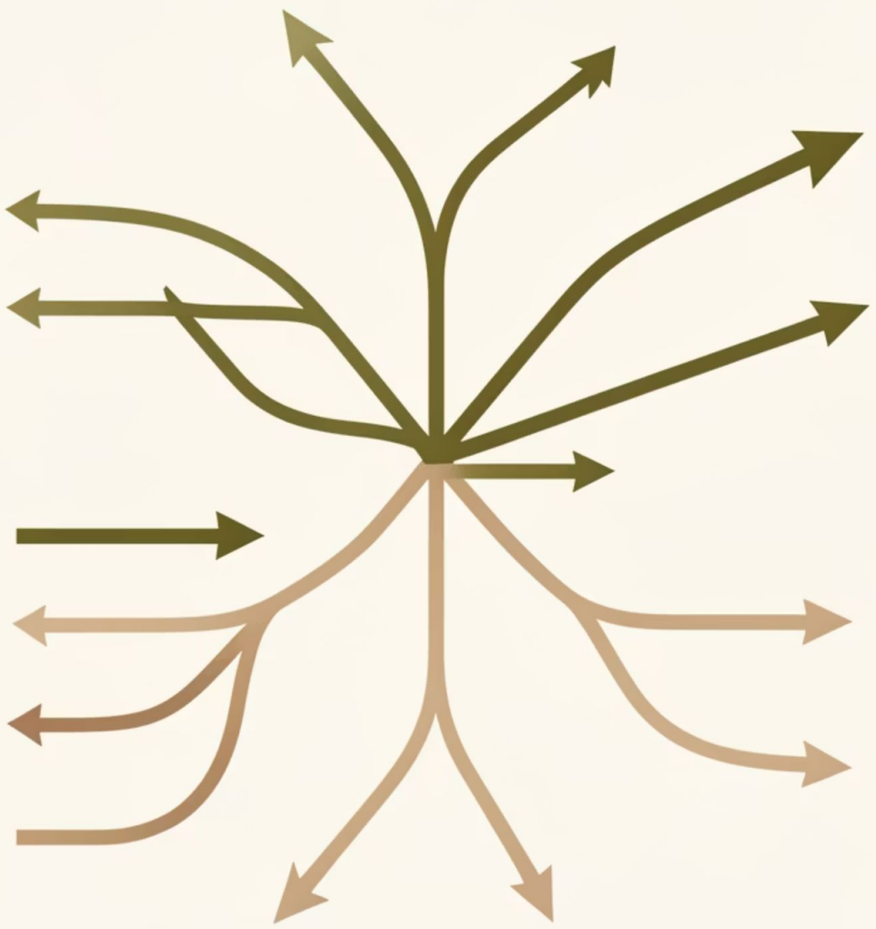
Cambia il branch attivo. Passa da un ramo all'altro in un secondo!

git switch -c

Crea un nuovo branch e ci si sposta immediatamente. La scorciatoia perfetta!

Suggerimento: In versioni più vecchie di Git, usa `git checkout` invece di `git switch`. Il comando `git switch` è più chiaro e moderno!

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it



Creare e Navigare tra Branch

Creazione di un Branch

Per creare un nuovo branch usa il comando `git branch` seguito dal nome che vuoi dare al branch.

```
git branch lezione-2
```

Questo crea un nuovo branch ma **non** ci sposta automaticamente. Per vedere tutti i branch disponibili:

```
git branch -a
```

- Il branch corrente sarà evidenziato con un asterisco (*)
- Verde indica il branch attivo
- Bianco indica gli altri branch

Cambiare Branch

Metodo moderno:

```
git switch lezione-2
```

Metodo classico:

```
git checkout lezione-2
```

Scorciatoia utile - crea e passa subito al nuovo branch:

```
git switch -c lezione-2
```

Git switch è più chiaro e specifico per cambiare branch. Usa sempre questo comando per i nuovi progetti.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Visualizzare la Storia: git log --graph

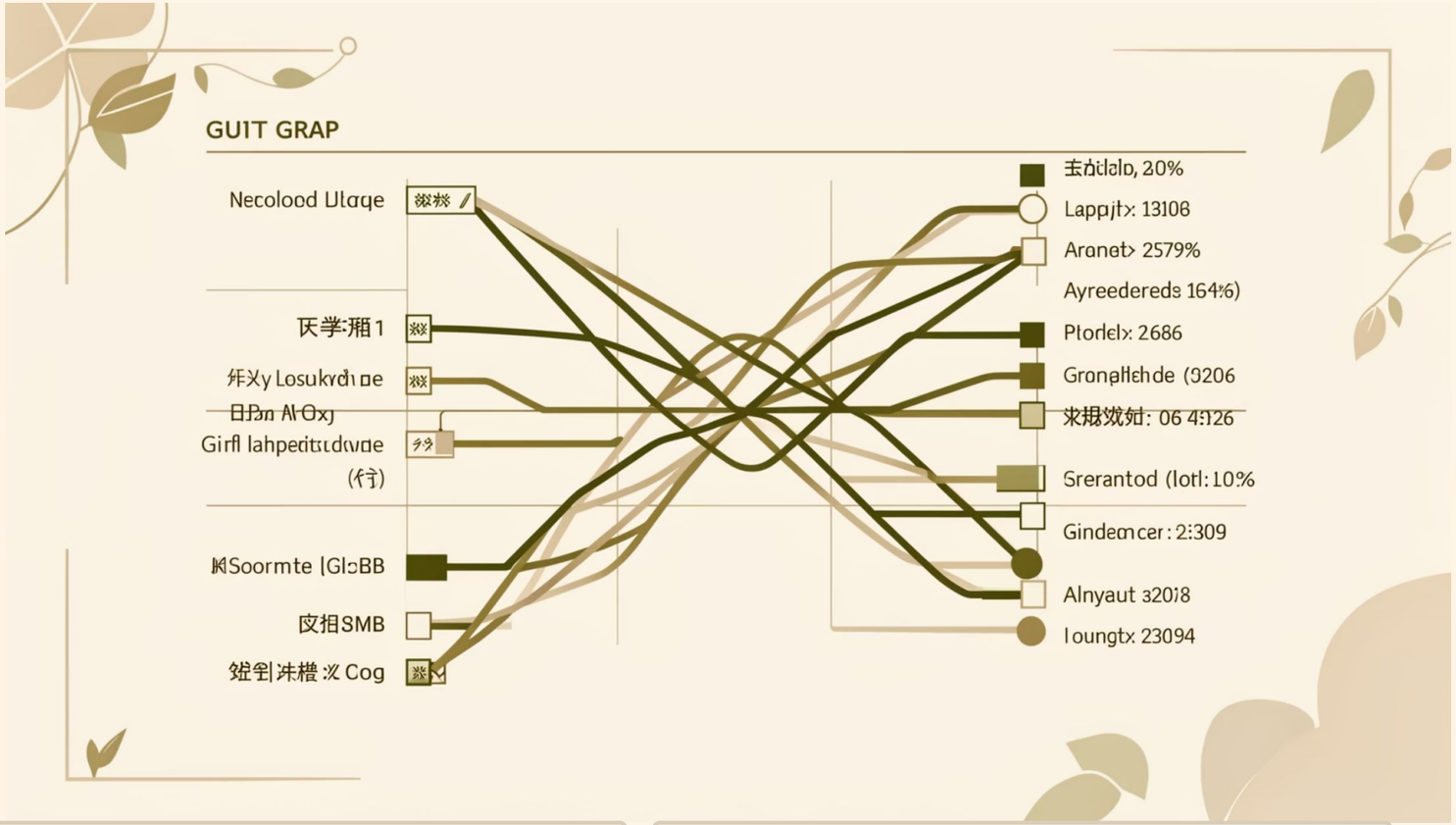
Il Comando Magico

Usa `git log --graph` per vedere la storia visiva del tuo progetto con tutte le diramazioni:

```
git log --graph --oneline
```

Questa comando ti mostra un bellissimo grafico ASCII che illustra come i branch si dividono e si uniscono nel tempo.

È come avere una mappa del tesoro del tuo progetto!



--graph

Aggiunge il diagramma visivo delle diramazioni

--oneline

Rende l'output più compatto e leggibile

--all

Mostra tutti i branch, non solo quello corrente

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Workflow Base: main + Feature Branch

Il pattern più comune e fondamentale per il lavoro con i branch.



Trunk stabile

Feature branch

Revisione & Test

Merge in main

Il Ramo Main

Rappresenta il codice **stabile e funzionante**

- È la versione che va in produzione
- Non si fa mai lavoro diretto qui
- Ogni commit dovrebbe essere testato e verificato

I Branch di Feature

Si creano da **main** per ogni nuova funzionalità

- Il lavoro avviene in isolamento
- Si possono fare molti commit sperimentali

Quando pronti, si uniscono a **main**

Questo workflow ti permette di lavorare su più funzionalità contemporaneamente senza conflitti!

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it



Best Practice: Come Chiamare i Tuoi Branch

Un nome ben scelto per i branch rende tutto più chiaro per te e i tuoi colleghi!

Usa nomi descrittivi e brevi

Evita nomi generici come "nuovo" o "prova". Scegli qualcosa che descriva esattamente cosa fa: **aggiungi-login**, **fix-header-bug**, **nuova-pagina-prodotti**

Usa trattini per separare le parole

Git preferisce trattini (-) invece di spazi o underscore: **carrello-acquisto** è meglio di **carrello_acquisto** o **carrello acquisto**

Evita caratteri speciali

Attieniti a lettere minuscole, numeri e trattini. Evita spazi, accenti, e caratteri come @, #, $ che possono causare problemi

Includi l'ID del ticket (se usi un sistema)

Se lavori con Jira o simili: **PROJ-123-fix-login-error** collega il branch direttamente al ticket di lavoro

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Esercizio Guidato Pratico

Ora è il momento di mettere in pratica quello che hai imparato. Segui questi passi:

Apri il Terminale

Naviga nella cartella del tuo progetto Git

Lista i Branch

Esegui `git branch` per vedere i branch esistenti

Crea un Nuovo Branch

Usa `git switch -c` per creare un branch chiamato **prova-branch**

Verifica la Posizione

Riesegui `git branch` per confermare che sei su **prova-branch**

Fai una Modifica

Crea un file di testo o modifica un file esistente nel tuo progetto

Commit delle Modifiche

Aggiungi e commit il file: `git add .` e `git commit -m "Test"`

Visualizza la Storia

Esegui `git log --graph --oneline` per vedere la diramazione

Torna a main

Completa con `git switch main` per tornare al branch principale

Bravo! Hai completato il tuo primo workflow con i branch. Continua a praticare questi comandi finché non diventano naturali!

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Git Merge e Gestione dei Conflitti

Una guida completa per principianti che hanno appena imparato i branch e vogliono padroneggiare l'unione delle modifiche nel loro workflow di sviluppo.

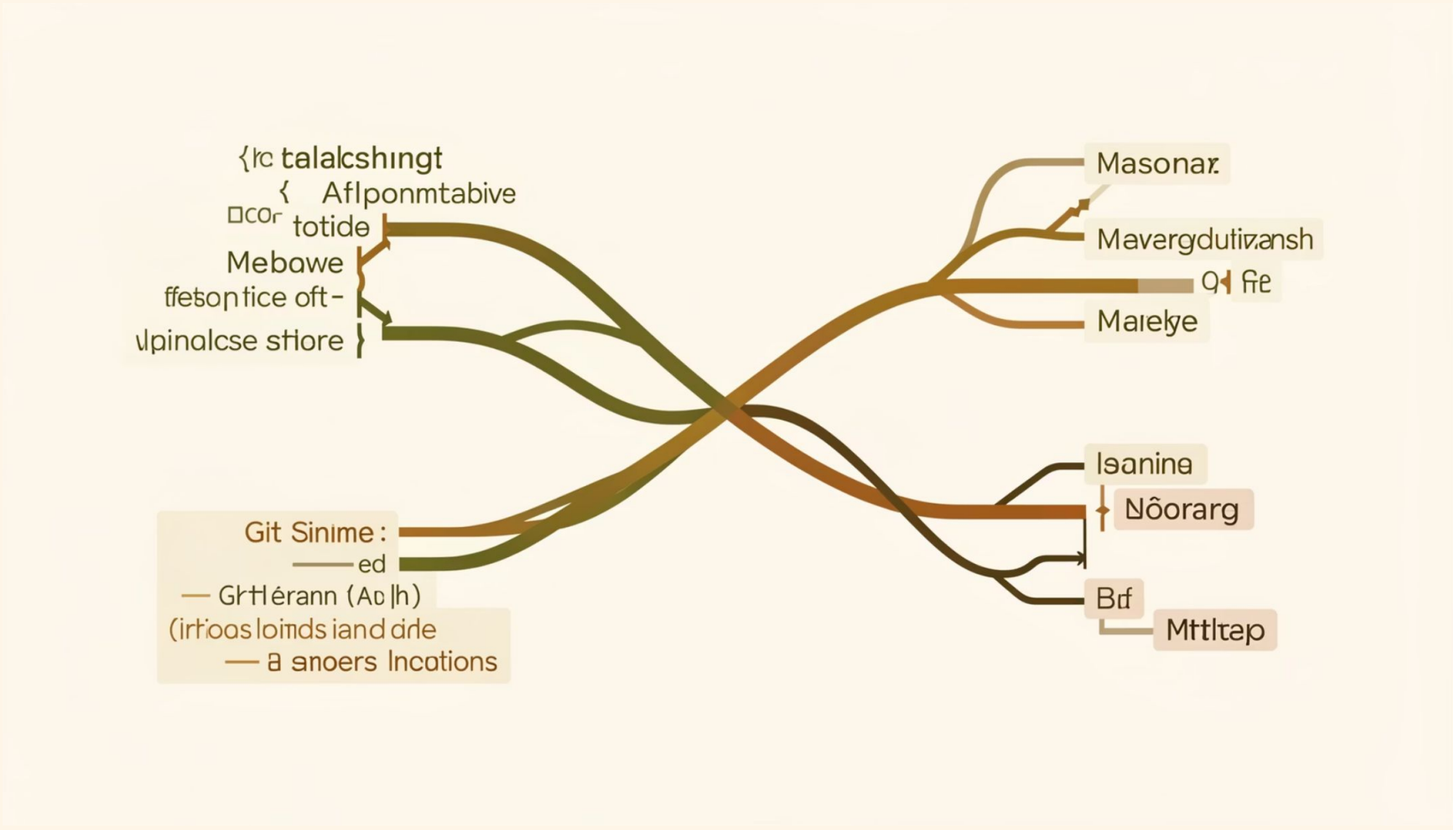


© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

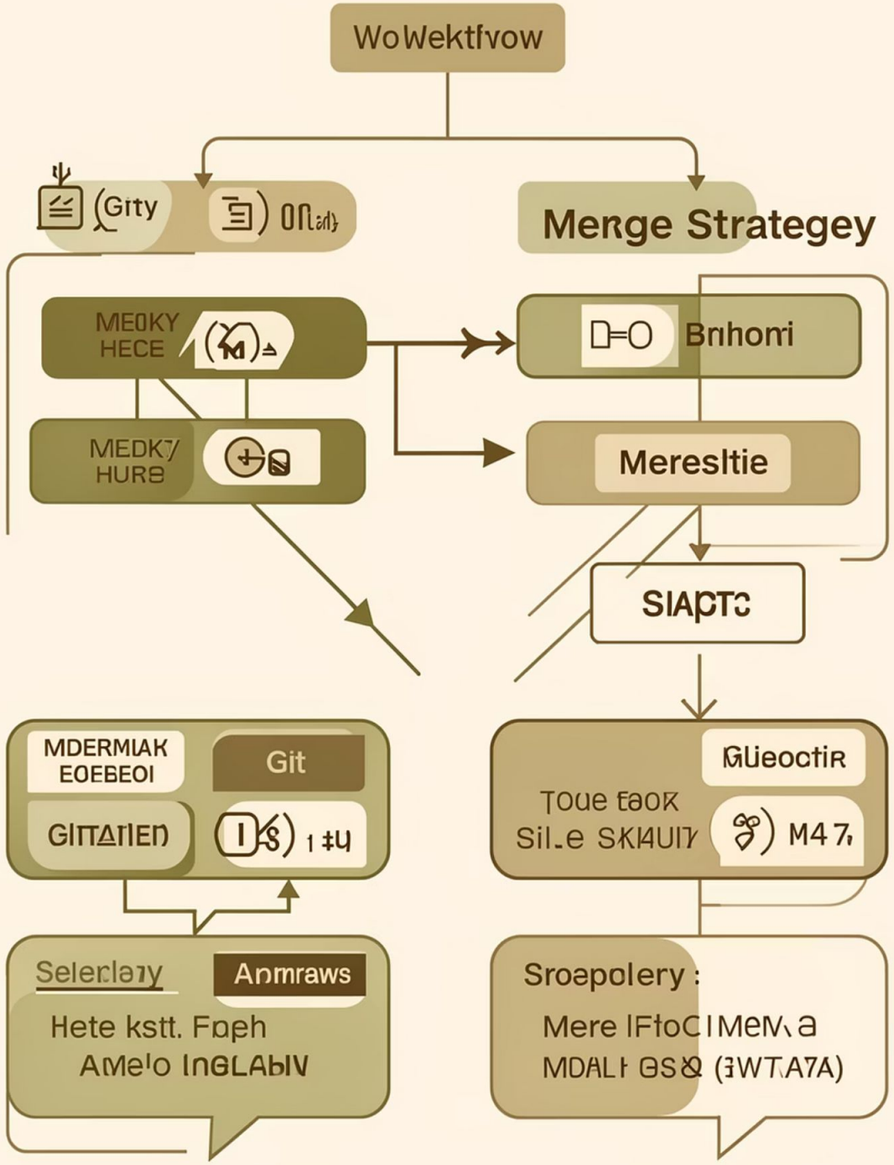
Cos'è il Merge in Git

Il merge è l'operazione fondamentale che permette di **unire le modifiche** da un branch a un altro. È come prendere tutti i cambiamenti sviluppati in una linea parallela di lavoro e incorporarli nella tua linea principale.
Pensa al merge come a un ponte che collega due strade separate, permettendo a tutti di passare da una strada all'altra senza perdere il percorso.

In Git, quando crei un branch, lavori su una copia isolata del tuo progetto. Il merge è il momento in cui integri quei cambiamenti nel branch principale (o in un altro branch).



© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it



Fast-Forward Merge vs Merge Commit

Fast-Forward Merge

Quando si verifica: Il branch di destinazione non ha nuove modifiche dall'ultimo commit

Come funziona: Git sposta semplicemente il puntatore del branch principale avanti lungo la cronologia

Risultato: Nessun commit di merge viene creato, la cronologia rimane lineare

Merge Commit

Quando si verifica: Entrambi i branch hanno modifiche divergenti

Come funziona: Git crea un nuovo commit speciale che unisce le modifiche di entrambi i branch

Risultato: Viene creato un commit di merge, la cronologia mostra la biforcazione

Nota: Il fast-forward merge è più pulito visivamente, ma il merge commit preserva la storia completa di come i branch si sono uniti.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Il Comando Git Merge

Sintassi di Base

```
git merge <nome-branch>
```

Questo comando unisce il branch specificato con il branch corrente. Prima di eseguire il merge, assicurati di essere sul branch destinazione.

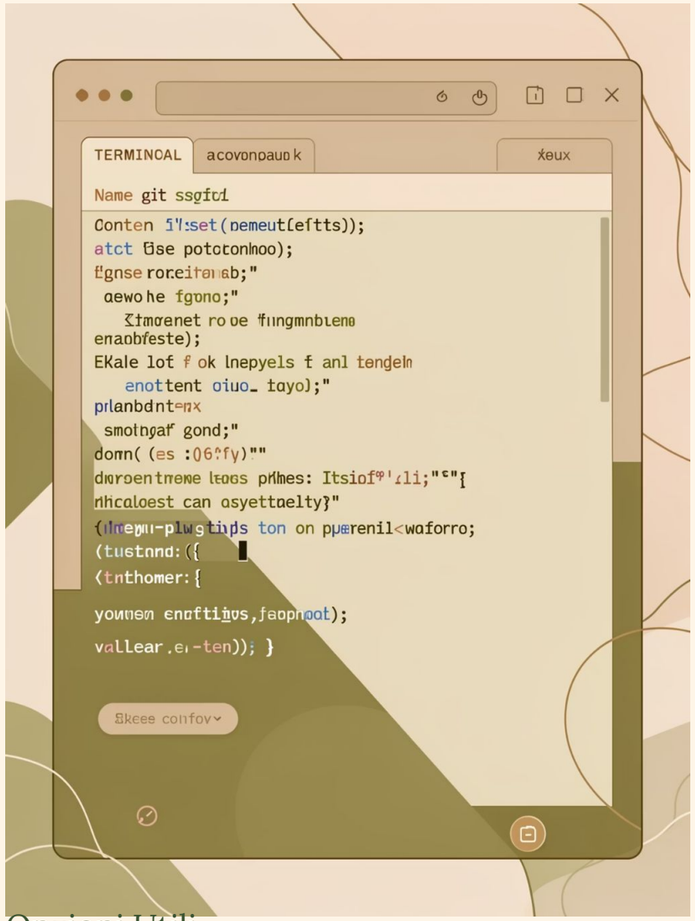
Passi Pratici

Passa al branch destinazione: `git checkout main`

Assicurati di essere aggiornato: `git pull origin main`

Esegui il merge: `git merge feature/login`

4. Risolvi eventuali conflitti
5. Commit delle modifiche unite



Opzioni Utili

`--no-ff`: Forza la creazione di un commit di merge

`--abort`: Annulla un merge in corso

`-m`: Aggiungi un messaggio personalizzato

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Simulazione Passo-Passo del Merge

Vediamo un esempio pratico di come si svolge un merge. Immaginiamo di avere un branch `main` e un branch `feature/news` con modifiche divergenti.

01

Configurazione Iniziale

Branch `main` a commit A, branch `feature/news` creato da A

02

Sviluppo Parallelo

Main: commit A → B → C | Feature: commit A → D → E

03

Checkout Main

`git checkout main` - Passiamo al branch principale

04

Esecuzione Merge

`git merge feature/news` - Git cerca di unire le modifiche

05

Creazione Commit

Git crea un commit di merge F che unisce C ed E

06

Risultato Finale

Branch main ora contiene tutte le modifiche da entrambi i branch

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

I Merge Conflict: Cosa Sono



Un **merge conflict** si verifica quando Git non può automaticamente unire le modifiche perché due branch hanno modificato la stessa parte di un file in modo diverso.
È come se due persone avessero modificato la stessa frase in un documento contemporaneamente. Git non sa quale versione sia corretta, quindi ti chiede di decidere manualmente.

Quando Si Verificano

- Modifiche alla stessa riga di codice
- Rinomina di file in un branch e modifiche nell'altro
- Eliminazione di file modificati nell'altro branch
- Struttura diversa del codice

Attenzione: I conflitti non sono errori! Sono normali nel lavoro di team e richiedono solo un po' di attenzione per risolverli correttamente.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Esempio Reale di Conflitto

Ecco come appare un file con marker di conflitto. Git inserisce **marker speciali** per indicare le diverse versioni del codice.

```
<<<<<<< HEAD
function saluta() {
  console.log("Ciao, mondo!");
}
=======
function saluta() {
  console.log("Hello, world!");
}
>>>>>>> feature/inglese
```

<<<<<<< HEAD

Inizio della tua versione corrente (branch principale). Quello che c'è tra questa e `=======` è il codice presente nel tuo branch.

=======

Separatore centrale che divide le due versioni del codice. Aiuta a distinguere chiaramente le modifiche.

>>>>>>> feature/inglese

Inizio della versione del branch che stai mergiando. Il codice fino a `>>>>>>>` è quello del branch di origine.

Dopo aver risolto il conflitto, devi **rimuovere tutti i marker** e lasciare solo il codice scelto, poi fare commit delle modifiche.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

TC-Web Technology Consulting

Procedura Completa di Risoluzione

Identifica i Conflitti

Git ti avvisa che ci sono conflitti. Usa `git status` per vedere quali file sono in conflitto. I file saranno marcati come "unmerged".

Apri i File

Apri ogni file con conflitto in un editor di testo. Cerca i marker `<<<<<<<`, `=======` e `>>>>>>>` per trovare le sezioni problematiche.

Risolvi Manualmente

Scegli quale versione mantenere o crea una soluzione che combini entrambe. Elimina tutti i marker di conflitto. Assicurati che il codice funzioni!

Aggiungi e Commit

`git add <file>` per segnalare che il conflitto è risolto. Poi `git commit` per completare il merge. Git suggerisce automaticamente il messaggio.

Verifica

Testa il codice per assicurarti che funzioni correttamente. Esegui i test e verifica che tutto sia a posto prima di pushare le modifiche.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Best Practice per Evitare Conflitti

Branch Breve

Crea branch specifici e lavoraci per periodi brevi. Più lungo è il tempo tra la creazione del branch e il merge, maggiori sono le possibilità di conflitti.

Usa un Editor Diff

Strumenti come VS Code, IntelliJ o editor diff dedicati rendono la risoluzione dei conflitti molto più semplice con highlight e suggerimenti automatici.

Modifica File Minimi

Quando possibile, limita le modifiche a pochi file. Più file modifichi, più probabilità hai di entrare in conflitto con altri sviluppatori che modificano gli stessi file.

Test Prima del Merge

Assicurati che il tuo codice funzioni prima di fare merge. Questo evita di introdurre bug nel branch principale e semplifica il debugging se qualcosa va storto.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Esercizio Pratico Guidato

È il momento di mettere in pratica quanto hai imparato! Segui questi passi per un'esperienza hands-on con i merge e i conflitti.

Prepara l'Ambiente

Crea una nuova cartella, inizializza Git e crea un file `hello.txt` con "Hello, World!"

Crea i Branch

`git branch feature/italian` e `git branch feature/spanish`. Passa su ciascuno e modifica il file in "Ciao, Mondo!" e "Hola, Mundo!"

Esegui il Merge

Checkout su main, fai merge di feature/italian, poi prova a fare merge di feature/spanish. Vedrai il conflitto!

Risolvi il Conflitto

Apri il file, trova i marker, scegli una versione o crea "Hello, Ciao, Hola World!", rimuovi i marker e fai commit

Verifica

Usa `git log --oneline --graph` per vedere la cronologia. Assicurati che tutto sia corretto e che il merge sia completato

Bravo! Ora hai padroneggiato i merge e la gestione dei conflitti. Continua a praticare e questi concetti diventeranno naturali nel tuo workflow quotidiano.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Esercizio 2: Branch e Merge Completo

1

Crea Branch Feature

```
git switch -c aggiungi-contatti
echo "Email: info@esempio.it" > contatti.txt
git add contatti.txt
git commit -m "Aggiunti contatti"
```

2

Torna a Main e Modifica

```
git switch main
echo "Versione 1.0" > versione.txt
git add versione.txt
git commit -m "Aggiunta versione"
```

3

Fai il Merge

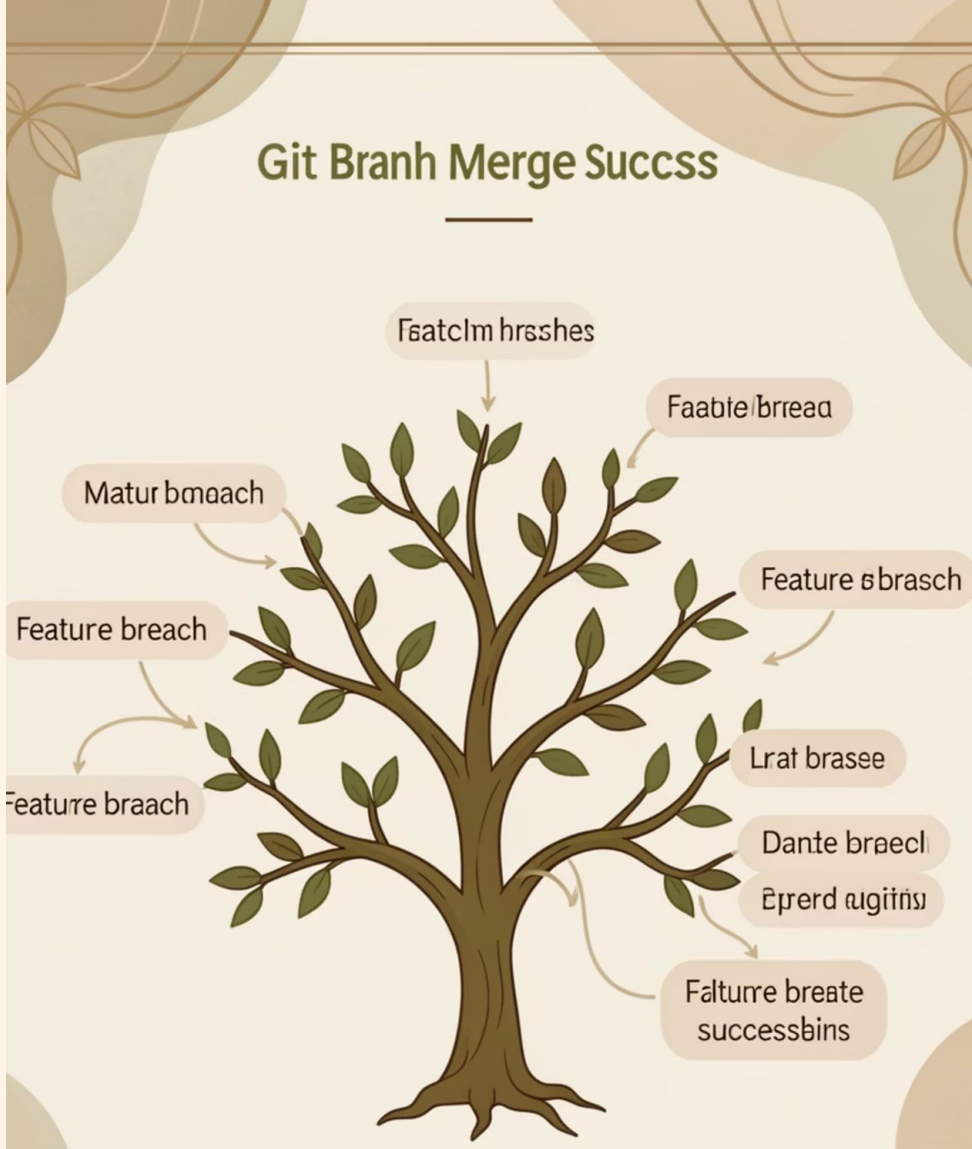
```
git merge aggiungi-contatti
git log --oneline --graph --all
```

4

Verifica Risultato

```
ls -la
cat contatti.txt
cat versione.txt
```

Soluzione completa: Dovresti vedere entrambi i file (contatti.txt e versione.txt) nel branch main. Il grafico dei log mostrerà due linee che si uniscono, rappresentando il merge del branch aggiungi-contatti nel main.



© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

GRAZIE A TUTTI…..

Via Schiaparelli 14 - 10148 Torino

https://www.tc-web.it

Mail: franco.grivet@tc-web.it

© 2026 Mattia Nardulli – Tutti i diritti riservati.